tcs AI
AI Fridays

TATA

Battle of the **best**

MENTOR GUIDE

Essential Guide for AI Friday National Finals Mentors

NATIONAL FINALS · 29 AUGUST – 4 SEPTEMBER 2026

AI FRIDAYS

National Finals · Season 2

TCS Internal

29th August – 4th September 2026

Version 1.0

Stay in a coaching role - enabling, not directing; questioning, not deciding; and never building on behalf of the team.

🎯 1. Purpose & Principles

This guide clarifies how mentors can maximize team learning and production-ready outcomes during the AI Friday National Finals Season 2 while strictly staying in a coaching role - enabling, not directing; questioning, not deciding; and never building on behalf of the team.

Coach, don’t lead
You are a sounding board and guide, not a decision-maker or product owner.

No hands-on building
Do not write code, design system components, or create assets for the team.

Foster autonomy
Encourage teams to choose approaches and own trade-offs.

Ask-first stance
Use questions to help teams discover solutions (Socratic mentoring).

Neutrality & fairness
Treat all teams equally; avoid favoritism or influence on judging.

Ethics & compliance
Keep data, access and usage within approved policies; prevent risky shortcuts.

2. Roles & Responsibilities

Before the event

Read the National Finals handbook and assigned problem statements; understand the production-ready expectations, constraints, deliverables, and evaluation criteria.

Confirm approved tools, platform access, support channels, and escalation paths for the National Finals.

Align the mentoring cadence, team touchpoints, office hours, and escalation protocol with the National Finals command center.

Prepare framework-agnostic coaching checklists and evaluation rubrics for guidance only; do not provide solution designs or implementation artifacts.

During the event

Run time-boxed mentoring touchpoints (15–30 mins per team) focused on business clarity, risks, assumptions, architecture choices, integration, evidence, and the next best actions.

Facilitate unblocking: connect teams to approved documentation, examples, platform support, or domain experts — never build or solve on behalf of the team.

Promote production engineering discipline: version control, modular design, testing, synthetic data quality, security, observability, reliability, deployment readiness, and documentation.

Monitor ethics/compliance: flag issues early and escalate to the command center if needed.

Prepare teams for final evaluation: sharpen the storyline, end-to-end working demo, business evidence, measured outcomes, architecture rationale, resilience, and production-readiness narrative.

After the event

Offer constructive, specific feedback focusing on decisions, learning, and next steps.

Capture top learnings and patterns to share with the community.

Capture follow-up recommendations for production adoption, scale-up, and continued solution maturity after the National Finals.

3. Domain Context & TCS Expert Connect

Mentors should help teams build enough contextual understanding of the assigned business domain to make informed decisions without solving the use case for them. Where deeper expertise is needed, connect the team with the right TCS domain, process, technology, risk, or industry experts.

What mentors should enable

Help the team understand the business process, key actors, terminology, decision points, pain points, and expected outcomes relevant to the use case.

Surface the domain assumptions that could materially affect the solution, such as workflow rules, exception paths, controls, regulatory considerations, service levels, or business KPIs.

Guide the team toward the right questions rather than prescribing the answer or selecting the business solution on their behalf.

Encourage teams to validate their understanding with evidence and clearly distinguish confirmed domain facts from assumptions.

TCS expert-connect approach

1 · Identify the gap
Pinpoint the specific domain question that the team cannot confidently answer from the problem statement, approved material, or their own research.

2 · Frame the ask
Help the team formulate a focused question with the relevant use-case context, assumptions, and decision they need to make.

3 · Connect to the right SME
Use the organizer / command-center channel to connect the team with an appropriate TCS domain or functional expert.

4 · Let the team own the outcome
The expert provides contextual guidance; the team remains responsible for interpreting it, making trade-offs, designing the solution, and implementing it.

Mentor guardrails for expert interactions

Do not ask experts to build code, create solution components, prepare deliverables, or make decisions that belong to the team.

Do not expose confidential customer information or bring unapproved data into the discussion. Use the approved problem context and synthetic / permitted data.

Keep expert conversations time-boxed and focused so they accelerate learning without becoming a substitute for team exploration.

✅ 4. Do & Don't

Recommended DOs

- Ask clarifying questions; surface assumptions and risks.
- Suggest alternative paths and trade-off frameworks (without deciding).
- Point to resources, patterns, and references.
- Ensure teams follow data governance and ethical AI practices.

Strict DON'Ts

- Don't write code, configure pipelines, or fix bugs for the team.
- Don't allocate tasks or set deadlines - that's the team's responsibility.
- Don't bring unapproved datasets, tools, or accounts into the environment.

⏱️ 5. Mentoring Cadence (National Finals · 7-day journey)

Kick-off clinic

~30 mins/team: Problem understanding, business outcomes, constraints, success metrics, architecture direction, and delivery plan.

Midpoint review

~20 mins/team: Working solution progress, evidence, blockers, integration risks, and next highest-value actions.

Pre-demo rehearsal

~20 mins/team: Production-ready solution storyline, end-to-end demo flow, measured outcomes, resilience, backup plan, and time-boxing.

6. Coaching Techniques

1. What evidence would change your current approach?
2. Which constraint is most binding right now, and how can we relax it?
3. What is the smallest working validation that can de-risk this assumption today while keeping the team on track for a production-ready solution?
4. How will you demonstrate measurable business and technical outcomes beyond the demo (quality, latency, cost, reliability, safety, and adoption)?
5. What’s your rollback/backup plan if the model or integration fails?